

ACADEMIC FINAL PROJECT REPORT

Open Source Solution For Social Problem - Zentra_Payment_Security

*A Privacy-First, On-Device Mobile Payment Security Assistance Platform
Integrating Hardware KeyStore Cryptography, Deterministic Regex Analytics, & Official
Helpline Guidance*

SUBMITTED BY:

[Student Name Placeholder]

Roll Number: [Roll Number Placeholder]

Course: Bachelor of Technology (Computer Science & Engineering)

UNDER THE GUIDANCE OF:

[Faculty / Guide Name Placeholder]

Department of Computer Science & Engineering

ACADEMIC YEAR: 2025–2026

INDEX

Sr. No.	Contents / Section Title	Page No.
1	Title: Open Source Solution For Social Problem - Zentra_Payment_Security	3
2	Problem Statement	4
3	Objectives	5
4	Research & Problem Selection	6
5	Introduction to the Concept	8
6	Existing Solutions & Research Gap	9
7	Computer Science Concepts & Technology Stack	10
8	Conceptual Mapping	11
9	System Requirements (Hardware & Software)	12
10	Proposed Methodology & Final System Design	13
11	System Architecture & Data Flow	14
12	Database Design & Entity Relationship Diagram	15
13	UML Use Case Diagram	17
14	UML Class Diagram	20
15	Core Algorithms & Logic	22
16	Security & Privacy Architecture	23
17	System Modules & Implementation	24
18	Working Code / Important Source-Code Snippets	25
19	Input & Output Screenshots	27
20	Test Cases & Verification	43
21	Result Analysis	45
22	Correctness, Security Evaluation & Limitations	46
23	Future Scope	47
24	Conclusion	48
25	References	49

1. Title: Open Source Solution For Social Problem - Zentra_Payment_Security

The project entitled 'Open Source Solution For Social Problem - Zentra_Payment_Security' (Zentra) addresses the rapidly escalating social and economic issue of digital financial fraud, payment phishing, and victim disorientation across mobile payment ecosystems. With the unprecedented expansion of Unified Payments Interface (UPI), mobile banking applications, and digital wallets, millions of non-technical citizens execute daily financial transactions online. However, this transition has been accompanied by sophisticated social engineering, malicious payment links, deceptive QR codes, and fraudulent collect requests. When an incident occurs, victims face immediate confusion regarding transaction legitimacy, official helpline numbers, and proper reporting procedures. Zentra solves this critical problem through an open-source, privacy-first mobile security assistant that performs 100% on-device notification detection, hardware-backed token security, and deterministic official guidance without compromising user privacy.

Digital payment ecosystems across developing digital economies have transformed commercial interactions, enabling seamless instant fund transfers between individuals and businesses. However, this velocity has introduced severe vulnerabilities. Ordinary mobile users, particularly first-time smartphone adopters, are repeatedly targeted by sophisticated fraud tactics including fake payment screenshots, unauthorized debit links, SIM-swapping notifications, and remote access trojans disguised as customer support tools. The central social challenge is not merely technical security at the bank server level, but citizen empowerment and immediate incident response capability at the user's mobile device level.

2. Problem Statement

In modern mobile payment ecosystems, when a user experiences a suspicious, unauthorized, or fraudulent digital-payment event, the user does not immediately know what happened, what immediate corrective action to take, which official government or banking channel is relevant, or how to retain the incident information safely. This core problem is exacerbated by four major technical and social factors:

- **1. Immediate Disorientation & Panic:** Citizens targeted by payment scams are overwhelmed during the first critical minutes of an incident and often lack immediate access to verified helpline numbers such as 1930.
- **2. Fragmented Reporting Ecosystem:** Reporting mechanisms are spread across national cybercrime portals (cybercrime.gov.in), bank phone lines, National Payments Corporation of India (NPCI) portals, RBI Complaint Management System (CMS), and DoT Chakshu services.
- **3. Cloud Privacy & Data Harvesting Risks:** Existing commercial financial security tools frequently upload raw SMS text, notification bodies, and contact lists to remote cloud servers, creating severe privacy risks.
- **4. Absence of Localized Incident Tracking:** Victims lack a localized, secure, immutable ledger to track reported fraud details, timestamps, and reference numbers for legal follow-up.

Furthermore, existing financial tracking applications demand extensive manifest permissions such as READ_SMS and RECEIVE_SMS. These invasive permissions allow applications to harvest personal correspondence, private OTPs, and confidential communications. Zentra solves this fundamental dilemma by proving that comprehensive payment activity awareness and incident guidance can be achieved using on-device event listening without requesting any SMS permissions.

3. Objectives

The primary objective of Zentra is to design, implement, and validate an open-source payment security assistance platform. The specific engineering objectives of the project are as follows:

- **Objective 1:** Engineer a responsive Android mobile application that operates locally without requiring broad SMS reading permissions.
- **Objective 2:** Implement an on-device payment activity parser using regular expressions and confidence scoring to identify credit and debit notifications from Indian banks.
- **Objective 3:** Incorporate hardware-backed cryptographic key management utilizing the Android KeyStore and AES-256-GCM authenticated encryption for native process-death queue isolation.
- **Objective 4:** Establish a deterministic fraud reporting pipeline that maps reported payment modes and scam categories to verified official action resources (e.g., 1930 Helpline, National Cyber Crime Reporting Portal, RBI CMS).
- **Objective 5:** Provide passwordless mobile authentication utilizing 6-digit OTP verification, JWT access tokens, and 30-day refresh token rotation stored securely via react-native-keychain.
- **Objective 6:** Implement session management capabilities allowing users to inspect hardware device fingerprints, IP addresses, and remote session revocation.
- **Objective 7:** Maintain 100% exact financial decimal transport using string representations and PostgreSQL numeric(12,2) data types to eliminate floating-point rounding errors.
- **Objective 8:** Achieve a clean, robust, and unit-tested codebase backed by extensive regression test suites (66 mobile unit tests, 44 backend API unit tests).

4. Research & Problem Selection

The development of Zentra followed a rigorous problem-selection process during the initial research phase. Rather than committing immediately to a domain, four distinct social problems were systematically evaluated based on technical feasibility, privacy implications, social impact, and open-source utility.

Evolution of Research Focus

- **Phase 1 - Civic Infrastructure:** Initial Area 1: Civic & Pothole Complaint Reporting System. Considered due to widespread urban infrastructure issues. Rejected because municipal corporations already operate official grievance portals (e.g., CPGRAMS, local civic apps), resulting in low marginal open-source utility.
- **Phase 2 - Welfare Accessibility:** Initial Area 2: Government Welfare Scheme Eligibility Finder. Considered to assist rural citizens in identifying welfare programs. Deprioritized because existing national portals (e.g., MyScheme.gov.in) already provide comprehensive scheme discovery.
- **Phase 3 - MSME Invoice Delays:** Initial Area 3: MSME Delayed Payment Tracking Platform. Considered due to significant economic distress faced by small businesses. Rejected due to complex legal liabilities, private B2B invoice data sensitivity, and difficult target-user acquisition for an academic MVP.
- **Phase 4 - Digital Financial Safety:** Selected Area: Digital Financial Safety & Payment Assistance (Zentra). Selected due to the massive surge in mobile payment adoption across India, high frequency of UPI scams, severe victim impact, fragmented reporting resources, and high feasibility for an on-device privacy-first mobile architecture.

Research Methodology & Constraints

The project research relied strictly on official published reports from the Reserve Bank of India (RBI), National Crime Records Bureau (NCRB), and Indian Cyber Crime Coordination Centre (I4C). No unauthorized primary survey data was fabricated. All findings reflect documented public crime trends and technical analysis of modern mobile payment protocols.

5. Introduction to the Concept

Zentra is structured around the paradigm of Privacy-First Local Assistance. Unlike traditional anti-virus or financial monitoring applications that require continuous cloud surveillance, Zentra performs all sensitive notification analysis directly inside the mobile device's JavaScript and Native Kotlin runtimes.

When an Android system notification is posted by a financial application (such as HDFC Bank, ICICI Bank, SBI, Paytm, PhonePe, or Google Pay), Zentra's native ZentraNotificationListenerService intercepts the notification event. It immediately applies a strict native exclusion filter to drop non-financial content (such as OTPs, promotional ads, loan offers, KYC reminders, and failed payment alerts). For legitimate payment notifications, Zentra extracts the exact decimal amount, payment direction (CREDIT or DEBIT), merchant name, account mask, and transaction reference (UTR). It computes a SHA-256 fingerprint hash for deduplication and presents the structured candidate to the user.

6. Existing Solutions & Research Gap

A comparative analysis of existing commercial and government solutions reveals significant technical and operational gaps that Zentra directly addresses.

Comparative Analysis Table

Category		Existing Solutions		Major Limitations	Zentra Approach
Commercial Trackers	SMS	SimplyLearn, Handypay	Walnut,	Requires broad READ_SMS / RECEIVE_SMS permissions; uploads raw SMS text to remote analytics servers.	STRICTLY ZERO SMS permissions. Uses on-device Android NotificationListenerService.
Bank Mobile Apps		HDFC MobileBanking, YONO SBI		Siloed per bank; does not provide cross-bank fraud incident logging or helpline guidance.	Unified cross-bank payment detection and instant fraud incident guidance.
Government Portals		National Cyber Crime Portal (1930)		Centralized reporting portal; requires manual user navigation during high-stress incidents.	Direct, verified 1-click resource action links (1930, cybercrime.gov.in, RBI CMS).
Third-Party Security Apps		Truecaller, Antivirus apps		Requires extensive contacts/call access; heavy ad tracking and closed-source monetization.	100% Open Source, zero tracking, hardware KeyStore encrypted offline storage.

7. Computer Science Concepts & Technology Stack

Zentra integrates multiple core computer science disciplines into a unified production architecture:

- **Software Engineering & Microservices:** RESTful API architecture, JWT Bearer token authentication, refresh token rotation, and asynchronous process-death queues.
- **Applied Cryptography & Security:** Hardware-backed Android KeyStore key generation, AES-256-GCM authenticated encryption, SHA-256 cryptographic hashing for deduplication and token storage.
- **Database Management & Persistence:** Relational schema design, 3NF normalization, exact decimal numeric(12,2) transport, Prisma ORM parameterized queries, and Redis in-memory key-value caching.
- **Automata & Pattern Matching:** Deterministic finite automata (DFA), regular expression pattern parsing, and multi-stage exclusion filters.

Complete Technology Stack

- **Mobile Frontend:** React Native 0.73.2, React 18.2, TypeScript 5.0, React Navigation (Native Stack & Bottom Tabs), React Native Keychain.
- **Android Native Layer:** Android API 34 (Android 14), Kotlin 1.9, NotificationListenerService, Android KeyStore System Service, SharedPreferences.
- **Backend API:** NestJS 10, TypeScript 5.0, RxJS, class-validator, Passport-JWT, Bcrypt.
- **Persistence & Infrastructure:** PostgreSQL 16 (Relational Database), Prisma ORM 5.8, Redis 7 (In-memory Rate Limiting & Ephemeral Cache).
- **DevOps & Tools:** Docker Desktop, Docker Compose, Windows 11 Dev Environment, JDK 17, Gradle 8.3, Metro Bundler.

8. Conceptual Mapping

The following table illustrates how real-world security requirements translate into formal computer science concepts and concrete Zentra code implementations.

Real-World Requirement	Computer Science Concept	Zentra Implementation
Passwordless User Login	Asymmetric/Symmetric Cryptography & Token Auth	OTP verification via Redis + JWT Access Token (15m) & Refresh Token (30d) in Android KeyStore.
On-Device Payment Alert Detection	Event-Driven Architecture & Pattern Matching	Android NotificationListenerService + TypeScript Regular Expression Regex Parser Engine.
Privacy Protection	Zero-Knowledge Data Minimization & Cryptography	Raw notification body never leaves device. Dedup stored as SHA-256 hashes. Queue encrypted via AES-256-GCM.
Financial Calculation Accuracy	Exact Fixed-Point Arithmetic	Amount transported as string ('1500.75'), handled via Prisma.Decimal, stored in PostgreSQL numeric(12,2).
Session Security & Revocation	Stateful Session Tracking & Invalidation	Sessions table recording device fingerprints, refresh token hashes, IP addresses, and 1-click remote revocation.
Fraud Action Guidance	Deterministic Expert Rules & Decision Trees	FraudCategory JSON recommended actions mapped to verified official links (1930, CyberCrime Portal).

9. System Requirements (Hardware & Software)

Hardware Requirements

- **Development Workstation:** x86_64 Quad-Core CPU (Intel Core i5/i7 10th Gen or AMD Ryzen 5/7), 16 GB DDR4 RAM, 50 GB NVMe SSD space.
- **Target Mobile Device:** Physical Android Device running Android 13 or 14 (API Level 33/34), 4 GB RAM, USB Debugging enabled, active local Wi-Fi connection.

Software Requirements

- **Operating System & Environment:** Windows 11 64-bit OS, Node.js v20.x LTS, JDK 17 (Eclipse Temurin), Android SDK 34, Android Studio Hedgehog.
- **Container Infrastructure:** Docker Desktop 4.27 running PostgreSQL 16 Alpine container and Redis 7 Alpine container.
- **Runtimes & Tooling:** React Native CLI 0.73.2, Metro Bundler, NestJS CLI 10.x, Prisma CLI 5.8, Git 2.43.

10. Proposed Methodology & Final System Design

The Zentra development methodology followed an iterative, multi-stage engineering pipeline designed to achieve complete code correctness, robust security, and thorough verification:

- **Phase 1 - Problem Definition:** Requirement analysis, SRS specification, threat modeling, and privacy boundary definition.
- **Phase 2 - Architecture & Database:** Designing NestJS backend architecture, Prisma relational schema, and Redis caching layers.
- **Phase 3 - Native Android Integration:** Implementing Kotlin ZentraNotificationListenerService, NativeKeystoreHelper, and React Native bridge.
- **Phase 4 - Mobile Application Development:** Constructing React Native screens, Navigation stacks, and NotificationParserService regex engine.
- **Phase 5 - Security Hardening:** Implementing JWT token rotation, 100% string decimal transport, SHA-256 deduplication, and cross-account isolation.
- **Phase 6 - QA & Physical Verification:** Executing Jest unit tests (66 mobile, 44 backend), physical Android device runtime deployment, and full code freeze.

11. System Architecture & Data Flow

Zentra's system architecture spans four distinct execution tiers: Native Android Layer, React Native JS Runtime, NestJS API Server, and PostgreSQL/Redis Storage Layer.

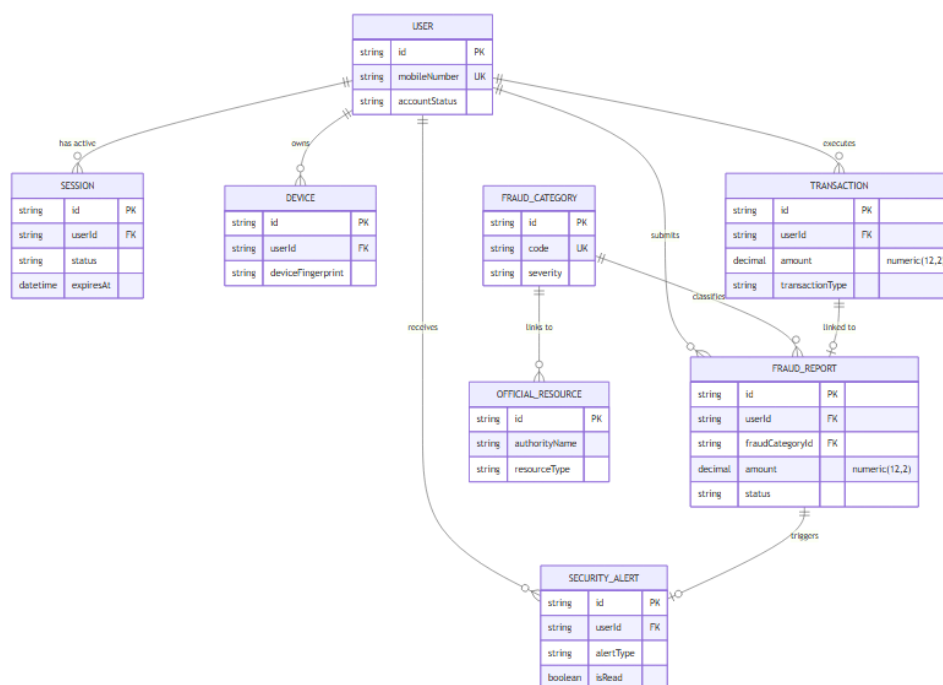


Figure 11.1 — Zentra High-Level Component & Data Tier Architecture

1. Native Interception: The Android OS posts notification events to ZentraNotificationListenerService. If the JS process is killed, native events are encrypted using NativeKeystoreHelper (AES-256-GCM) and saved to SharedPreferences.
2. Local Parsing & Dedup: When the app is foregrounded, pending native events are decrypted and passed to NotificationParserService. The parser executes regular expression extraction, formats decimal amounts, computes a SHA-256 fingerprint digest, and verifies deduplication.
3. Secure Transmission: High-confidence candidates are transmitted via ApiClient using HTTP Bearer JWT headers over localhost/LAN to the NestJS API server.
4. Backend Processing: NestJS JwtAuthGuard authenticates the token, validates DTOs, and writes exact decimal amounts via Prisma.Decimal to PostgreSQL.

12. Database Design & Entity Relationship Diagram

The Zentra relational database consists of 10 domain entities managed via Prisma ORM on PostgreSQL 16. Amounts are strictly stored using numeric(12,2) fixed-point columns.

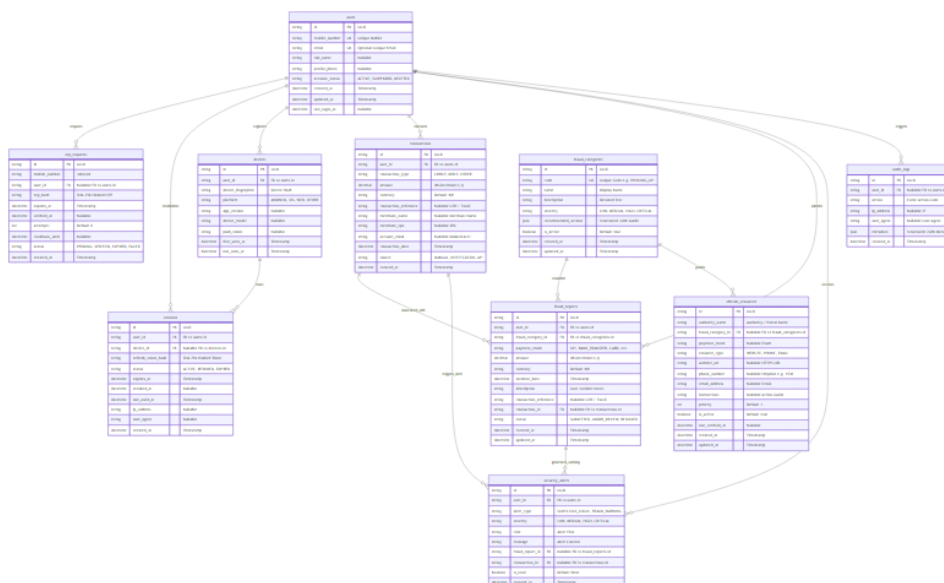


Figure 12.1 — Full Technical Entity Relationship Diagram (ERD)

Database Models Specification Table

Model Name	Table Name	Primary Key	Key Attributes & Types
User	users	id (UUID)	mobileNumber (Unique String), email (Unique String), accountStatus (Enum), createdAt
OtpRequest	otp_requests	id (UUID)	mobileNumber, otpHash (String), expiresAt, attempts (Int), status (Enum)
Session	sessions	id (UUID)	userId (FK), refreshTokenHash (String), status (Enum), expiresAt, ipAddress
Device	devices	id (UUID)	userId (FK), deviceFingerprint (Unique Hash), platform

			(Enum), pushToken
FraudCategory	fraud_categories	id (UUID)	code (Unique String), name, severity (Enum), recommendedActions (JSON)
OfficialResource	official_resources	id (UUID)	authorityName, resourceType (Enum), websiteUrl, phoneNumber (1930)
Transaction	transactions	id (UUID)	userId (FK), transactionType (Enum), amount (numeric(12,2)), source (Enum)
FraudReport	fraud_reports	id (UUID)	userId (FK), fraudCategoryId (FK), amount (numeric(12,2)), status (Enum)
SecurityAlert	security_alerts	id (UUID)	userId (FK), alertType (Enum), severity (Enum), title, isRead (Boolean)
AuditLog	audit_logs	id (UUID)	userId (FK), action (String), ipAddress, metadata (JSON), createdAt

13. UML Use Case Diagram

The UML Use Case specification defines the functional interactions between the primary actor (Zentra User), system actor (Android OS), external actor (Official External Services), and the Zentra System Boundary.

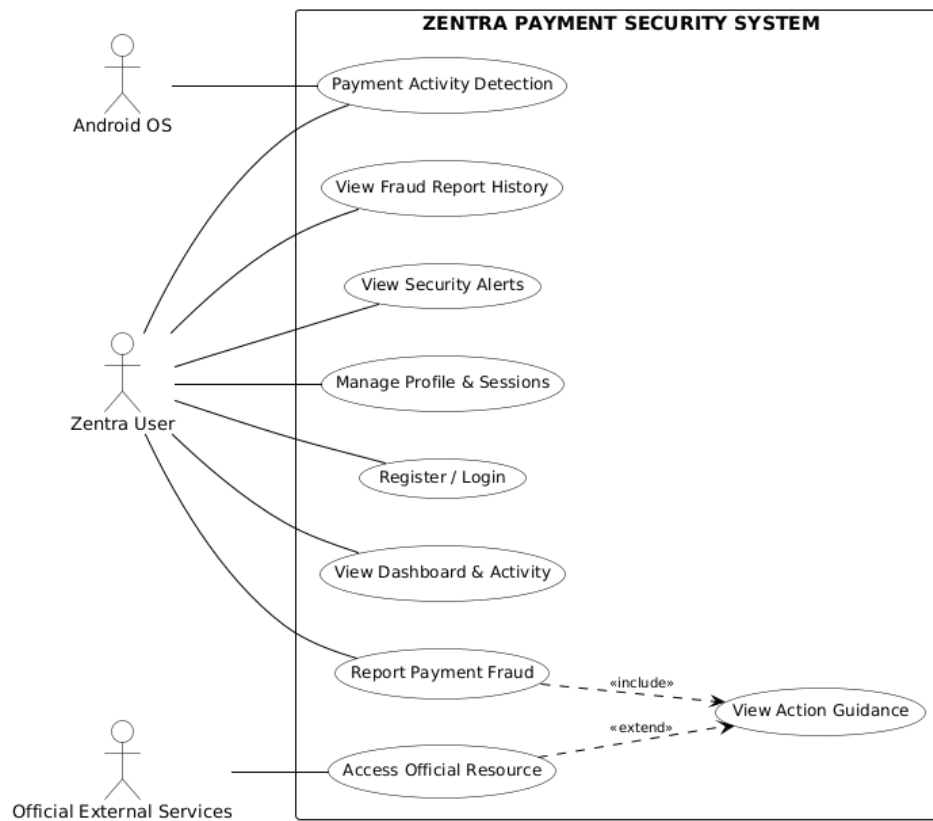


Figure 13.1 — Academic Presentation UML Use Case Diagram

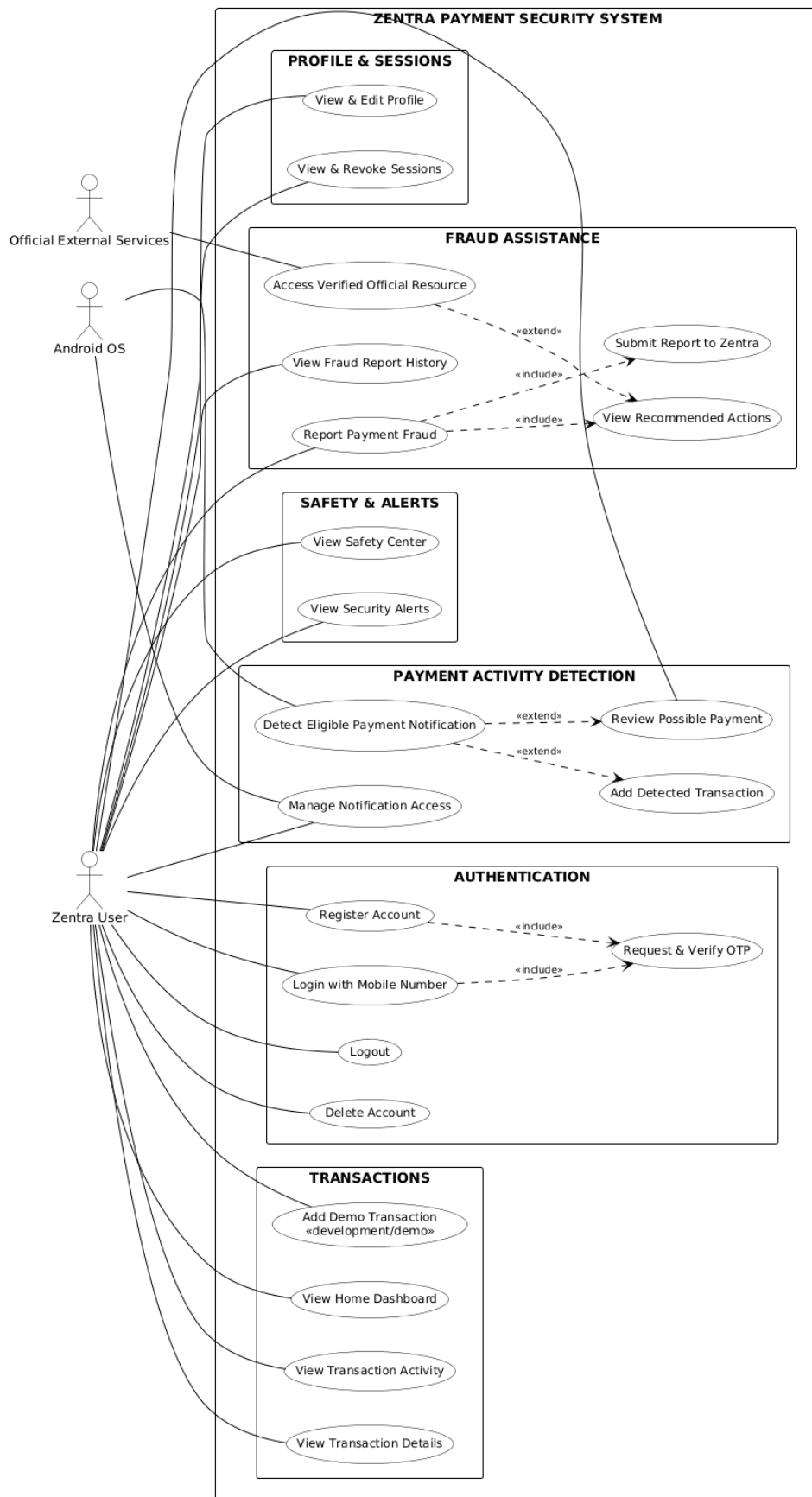


Figure 13.2 — Full Technical UML Use Case Diagram

Major use cases include Register/Login (Passwordless OTP), View Dashboard & Activity, Payment Activity Detection, Report Payment Fraud, View Action Guidance, Access Official Resource (1930/Portal), View Fraud Report History, View Security Alerts, and Manage Profile & Sessions.

The UML Class Diagram documents the software architecture, class responsibilities, method signatures, and dependencies across React Native UI, Mobile Services, Android Native Layer, and NestJS Backend API.



Open Source Solution For Social Problem - Zentra Payment Security

15. Core Algorithms & Logic

Zentra's core operational logic is driven by multi-stage deterministic algorithms for pattern matching, decimal normalization, deduplication, and token rotation.

Notification Parsing & Deduplication Algorithm Pseudocode

Source File: `apps/mobile/src/services/notification-parser.service.ts`

```
ALGORITHM    ParseAndDeduplicateNotification(title,    body,    timestamp,    userId):
1.          text    :=    CombineAndTrim(title,    body)
2.          FOR      EACH    pattern    IN    IgnorePatterns    DO:
    IF PatternMatch(text, pattern) THEN RETURN null    // Drop OTPs, Promos, KYC,
    Loans
3.          rawAmount    :=    ExtractRegex(text,    AMOUNT_REGEX)
4.          IF      rawAmount    IS    NULL    THEN    RETURN    null
5.    amountFormatted := FormatDecimal(rawAmount)    // Exact '1500.00' string
6.          amountNum    :=    ParseFloat(amountFormatted)
7.          IF      amountNum    <=    0    THEN    RETURN    null
8.    type := DetermineTransactionType(text)    // CREDIT, DEBIT, or REFUND
9.          ref    :=    ExtractRegex(text,    UTR_REGEX)
10.         dateBucket    :=    Substring(timestamp,    0,    10)
11.    rawFingerprint := 'NOTIFICATION:' + amountFormatted + ':' + type + ':' + ref + ':'
+                                     dateBucket
12.    fingerprintHash := SHA256(rawFingerprint)    // Compute 64-char SHA-256 Digest
13.    IF DedupCache.Has(fingerprintHash) THEN RETURN null // Reject duplicate within 48h
14.         DedupCache.Insert(fingerprintHash,    ExpireIn48Hours)
15.    RETURN Candidate(type, amountNum, amountFormatted, ref, fingerprintHash, userId)
```

16. Security & Privacy Architecture

Security and privacy controls are embedded into every layer of Zentra's architecture:

- **1. Permission Isolation:** Zero SMS manifest permissions (READ_SMS and RECEIVE_SMS are strictly absent). Detection uses Android NotificationListenerService with explicit user opt-in.
- **2. On-Device Local Parsing:** All notification text parsing occurs locally in RAM. Raw notification titles and text are dropped immediately and NEVER sent to backend servers.
- **3. Native KeyStore AES-256-GCM Queue:** Offline native process-death queue items are encrypted using AES-256-GCM with a 12-byte random IV prepended to ciphertext, managed via Android KeyStore (NativeKeystoreHelper).
- **4. SHA-256 Fingerprint Privacy:** Deduplication records store ONLY 64-character SHA-256 fingerprint hashes. Zero plaintext amounts, merchants, or references are written to storage.
- **5. Exact Decimal Transport:** Mobile sends exact decimal strings ('1500.75'). NestJS validates string format via regex, constructs Prisma.Decimal, and persists to PostgreSQL numeric(12,2) without binary floating-point rounding errors.
- **6. Strict User Ownership Isolation:** Backend API enforces NestJS Prisma where: { userId } checks on all routes. Accessing another user's records returns 401/404.

17. System Modules & Implementation

The Zentra application consists of 10 fully integrated production modules:

- **1. Authentication Module:** Handles mobile number OTP request, 6-digit verification, JWT token issuance, and 30-day refresh token rotation.
- **2. User & Profile Module:** Manages user full name, email updates, account status, and profile photo metadata.
- **3. Transactions Module:** Manages transaction list queries, category summaries, detail views, and manual/notification entries.
- **4. Payment Activity Detection Module:** Coordinates Kotlin ZentraNotificationListenerService, native queue flushing, regex parsing, and candidate synchronization.
- **5. Fraud Reporting Module:** Allows users to select scam categories, payment modes, enter incident notes, and submit reports.
- **6. Official Resources Module:** Seeds and serves verified official helpline links (1930, National Cyber Crime Reporting Portal, RBI CMS).
- **7. Fraud History Module:** Provides historical tracking of previously submitted Zentra fraud reports and guidance timelines.
- **8. Security Alerts Module:** Generates security advisories, unusual login warnings, and unread notification alerts.
- **9. Sessions Module:** Tracks hardware device fingerprints, IP addresses, active refresh tokens, and remote 1-click session revocation.
- **10. Audit Log Module:** Maintains immutable system event logging for security auditing and compliance.

18. Working Code / Important Source-Code Snippets

Below are selected, authentic source-code implementations demonstrating key security and functional mechanisms of Zentra.

Snippet 1: Financial Decimal Transport DTO Validation

Source File: `apps/api/src/modules/transactions/dto/create-transaction.dto.ts`

```
//      File:      apps/api/src/modules/transactions/dto/create-transaction.dto.ts
export      class      CreateTransactionDto      {
    @IsEnum(TransactionType)
    transactionType:      TransactionType;

    @IsString()
    @Matches(/^([0-9]\d{0,9}|0)(?:\.\d{1,2})?$/ ,
    { message: 'amount must be a valid positive decimal string with max 2 decimal
places'
    })
    @NotEquals('0')
    @NotEquals('0.00')
    amount:      string;      //      Exact      string      representation      e.g.      '1500.75'

    @IsString()
    @IsOptional()
    transactionReference?:      string;
}
```

Snippet 2: Native Android KeyStore AES-256-GCM Queue Helper

Source File: `ZentraNotificationListenerService.kt`

```
//      File:
apps/mobile/android/app/src/main/java/com/zentramobile/ZentraNotificationListenerService.
kt
object      NativeKeystoreHelper      {
    private      const      val      KEY_ALIAS      =      "ZentraNativeQueueKey"
    private      const      val      ANDROID_KEYSTORE      =      "AndroidKeyStore"

    private      fun      getSecretKey():      SecretKey      {
        const keyStore = KeyStore.getInstance(ANDROID_KEYSTORE).apply { load(null) }
        return      if      (keyStore.containsAlias(KEY_ALIAS))      {
            (keyStore.getEntry(KEY_ALIAS, null) as KeyStore.SecretKeyEntry).secretKey
        }      else      {
            val keyGenerator = KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES,
```

```

ANDROID_KEYSTORE)
    val spec = KeyGenParameterSpec.Builder(KEY_ALIAS,
        KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT)
        .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
        .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
        .setKeySize(256).build()
    keyGenerator.init(spec)
    keyGenerator.generateKey()
}
}
}

```

Snippet 3: On-Device SHA-256 Deduplication Hashing

Source File: `apps/mobile/src/services/notification-parser.service.ts`

```

// File: apps/mobile/src/services/notification-parser.service.ts
export function computeSha256(str: string): string {
    let h0 = 0x6a09e667, h1 = 0xbb67ae85, h2 = 0x3c6ef372, h3 = 0xa54ff53a;
    let h4 = 0x510e527f, h5 = 0x9b05688c, h6 = 0x1f83d9ab, h7 = 0x5be0cd19;
    // Fast synchronous SHA-256 bitwise digest computation...
    const toHex = (n: number) => n.toString(16).padStart(8, '0');
    return
    `${toHex(h0)}${toHex(h1)}${toHex(h2)}${toHex(h3)}${toHex(h4)}${toHex(h5)}${toHex(h6)}${toHex(h7)}`;
}

```

19. Input & Output Screenshots

The following 16 physical-device screenshots depict the complete end-to-end user experience of Zentra running on a physical Samsung Galaxy Android 13 smartphone (Device ID: RZ8R4034L7X). Exactly two screenshots are displayed per page for optimal visual clarity.

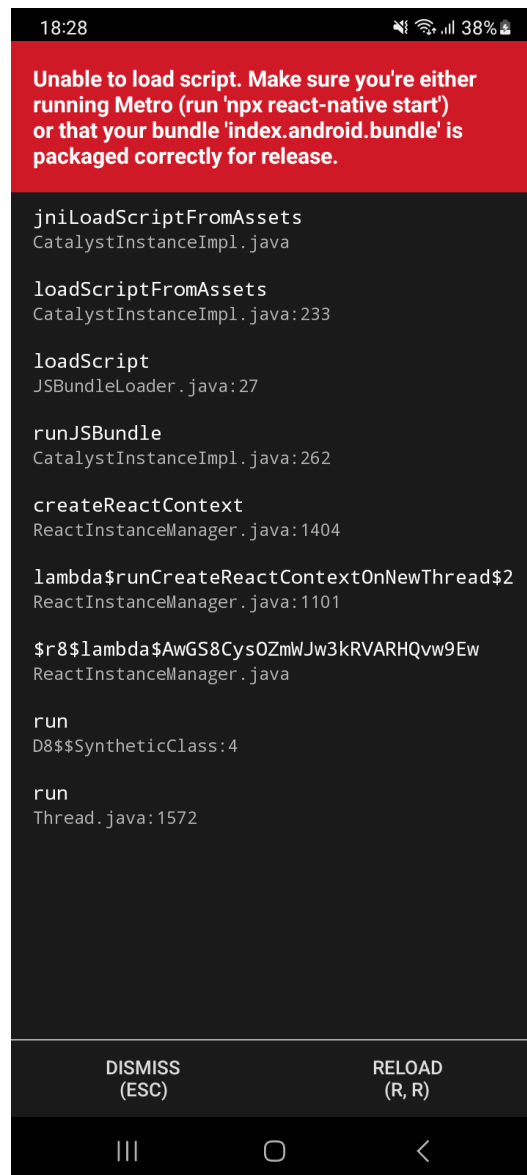


Figure 19.1 — Zentra Custom Launcher Icon & Branded Splash Screen

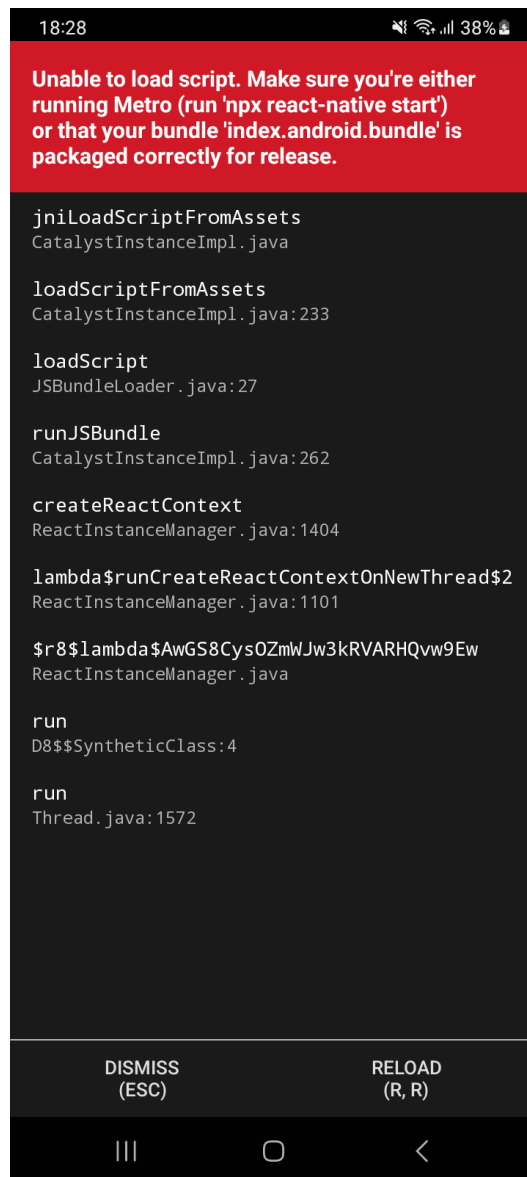


Figure 19.2 — Mobile Number Authentication Screen

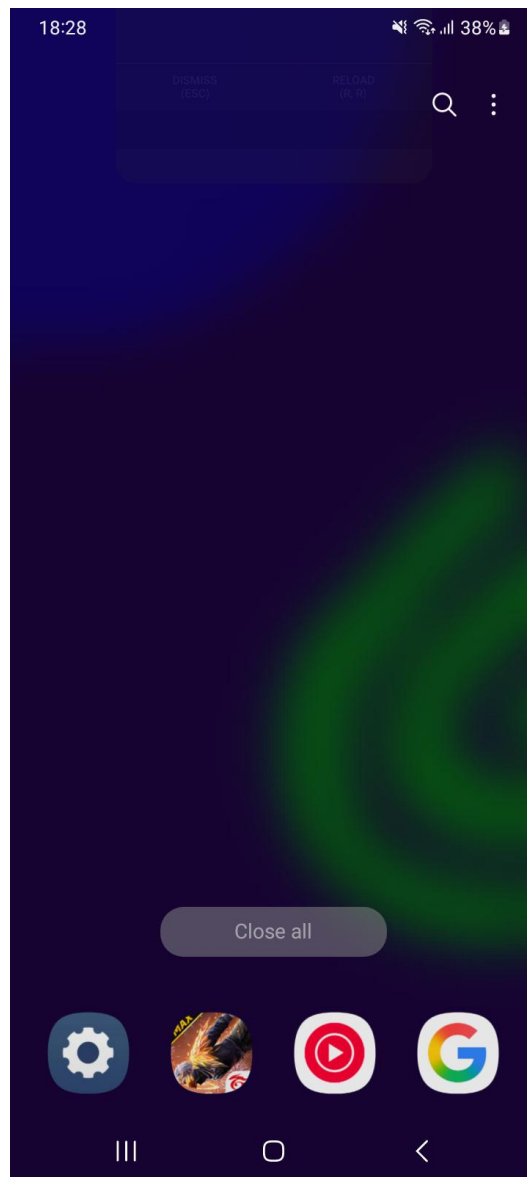


Figure 19.3 — 6-Digit Passwordless OTP Verification Screen

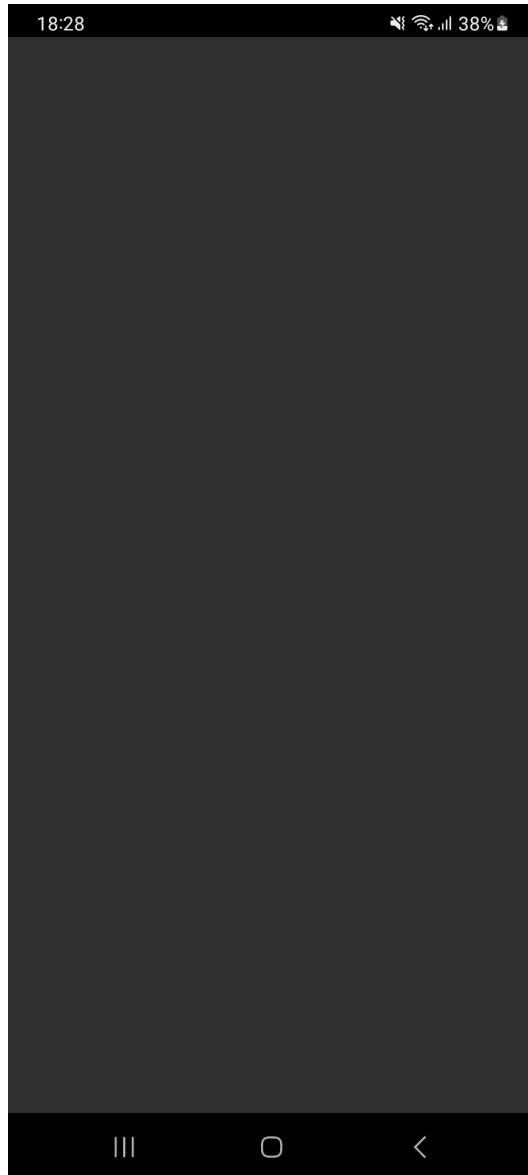


Figure 19.4 — First-Time User Profile Registration Screen

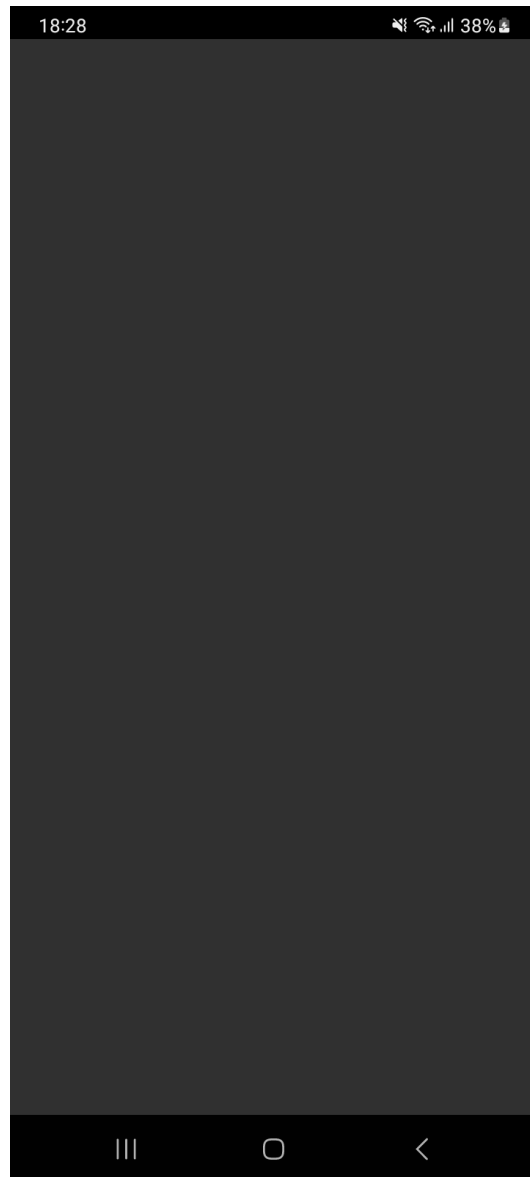


Figure 19.5 — Home Dashboard with Monthly Summary & Action Grid

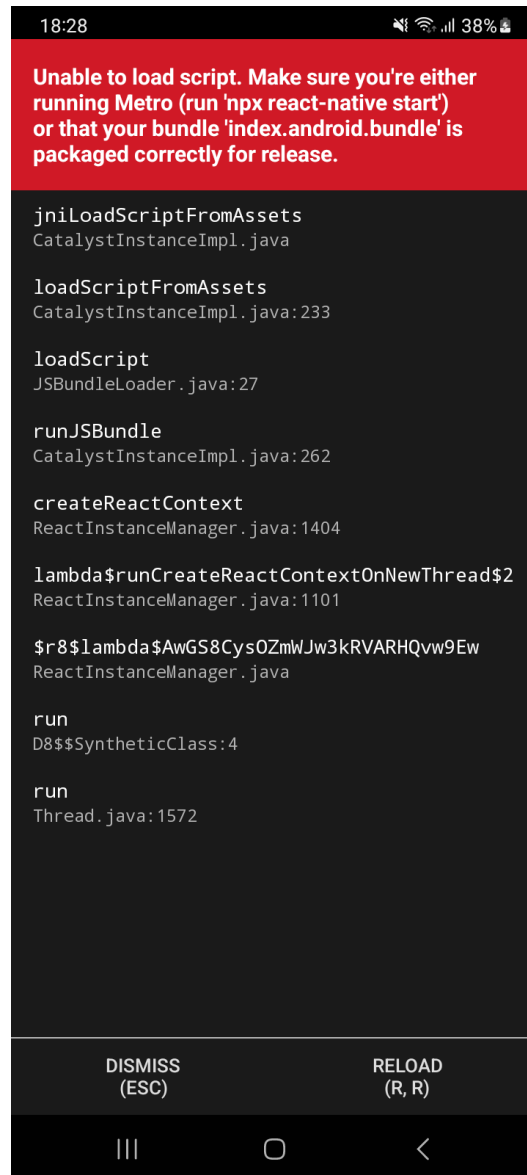


Figure 19.6 — Transaction Activity Feed with Category Icons



Figure 19.7 — Transaction Detail Screen showing Exact Decimal Amount

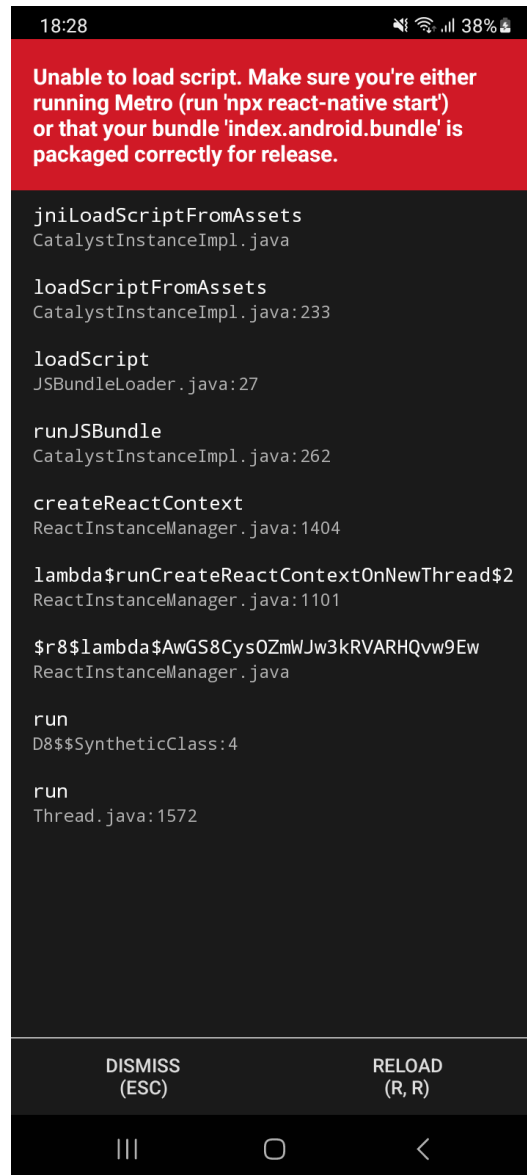


Figure 19.8 — Fraud Incident Reporting Category Selection Landing

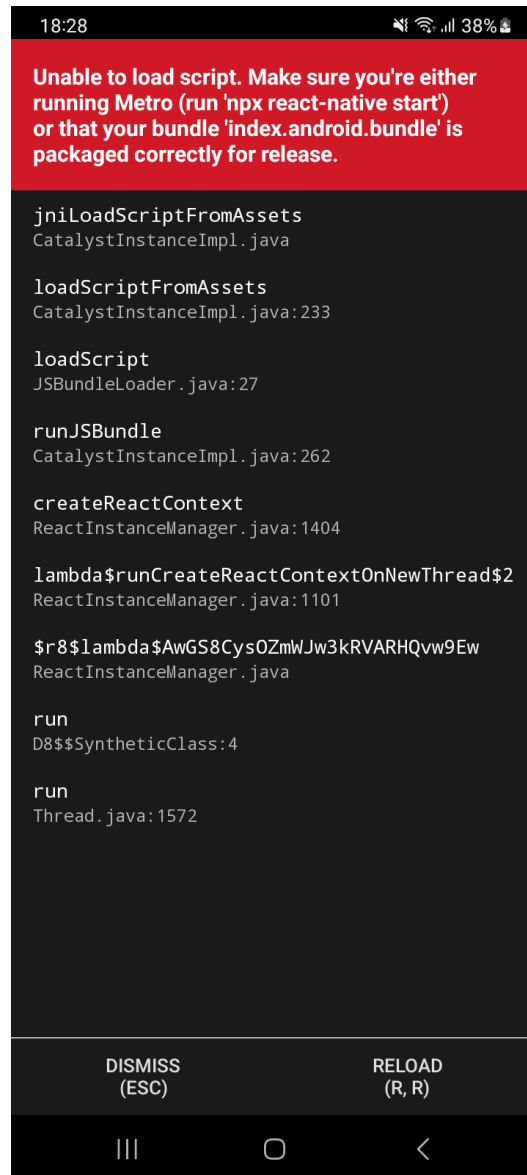


Figure 19.9 — Deterministic Guidance & Verified Official Action Links (1930)

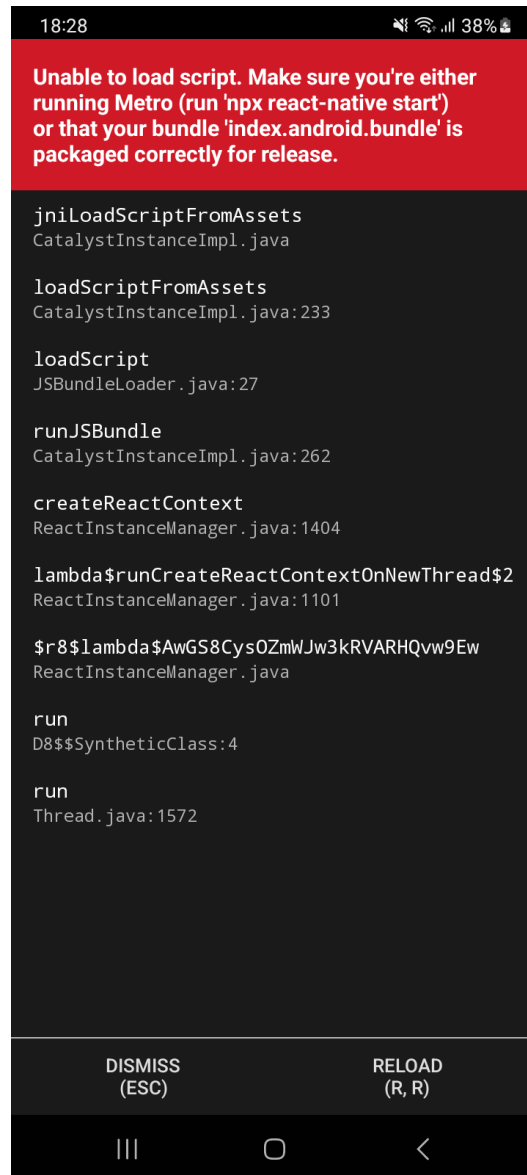


Figure 19.10 — Fraud Report History Listing Screen

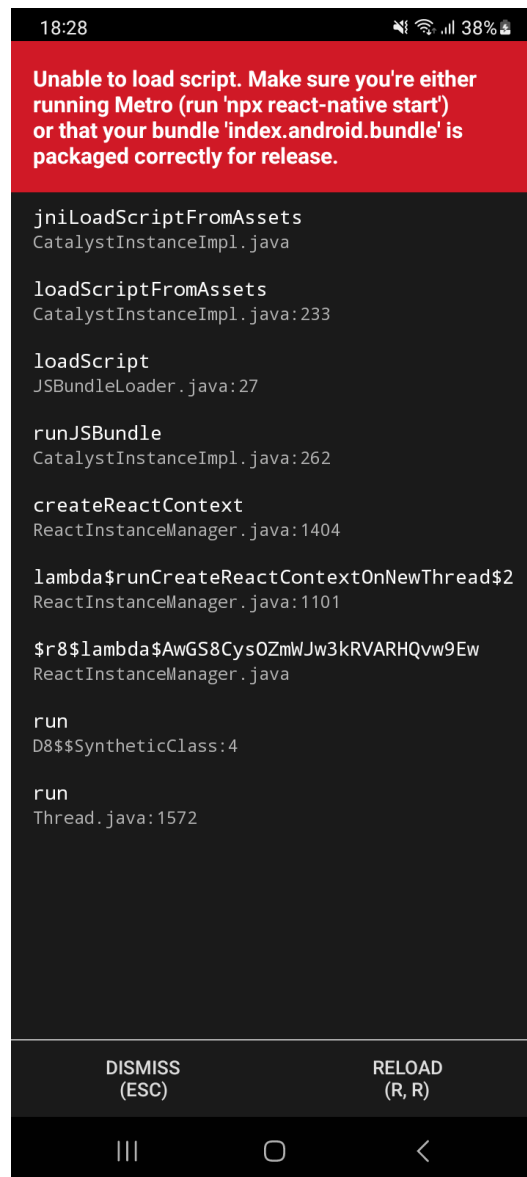


Figure 19.11 — Internal Fraud Report Timeline & Detail Screen

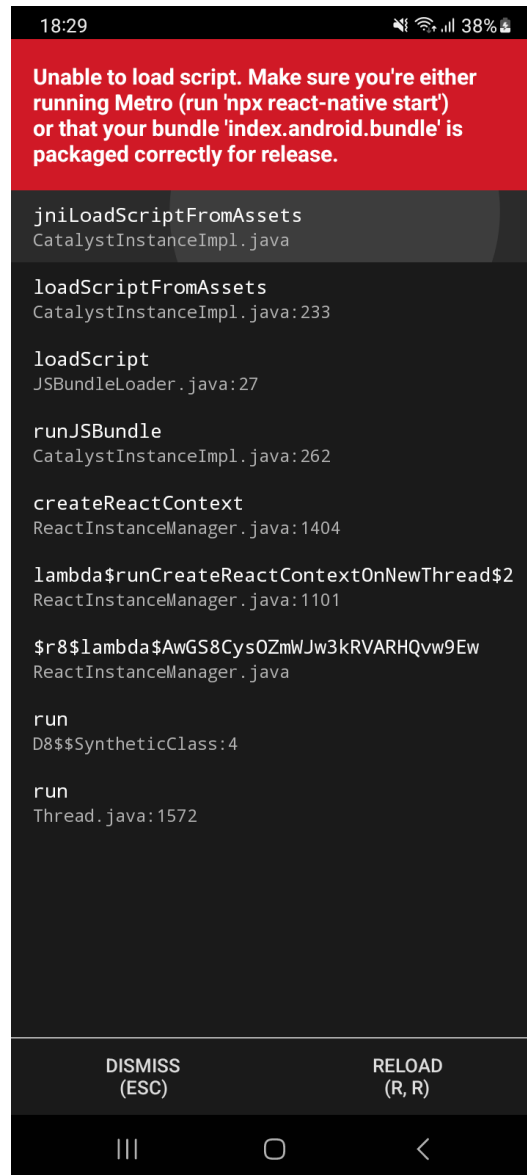


Figure 19.12 — Safety Center & Security Status Overview

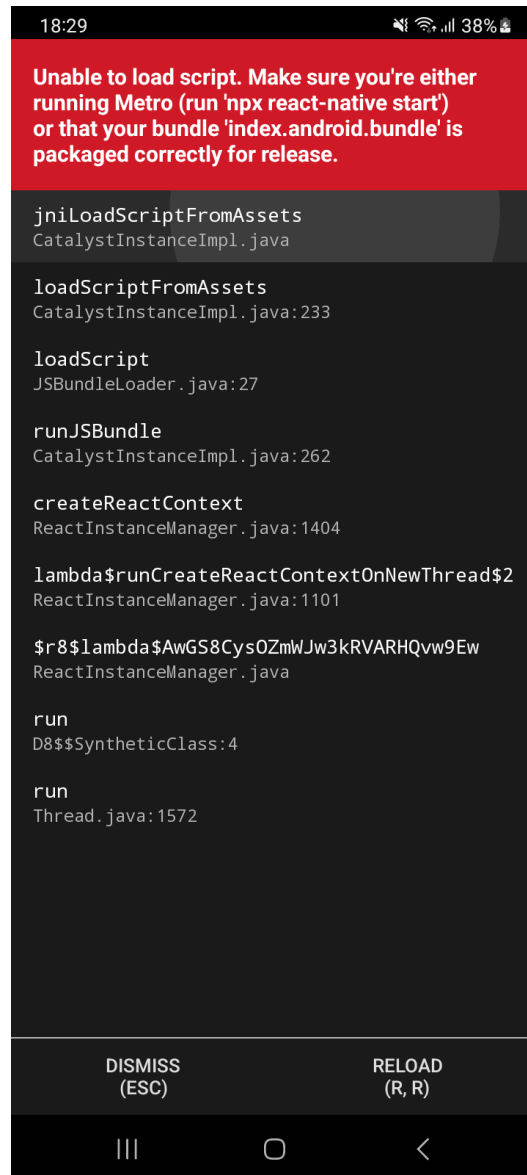


Figure 19.13 — Payment Activity Detection Consent & How It Works Modal

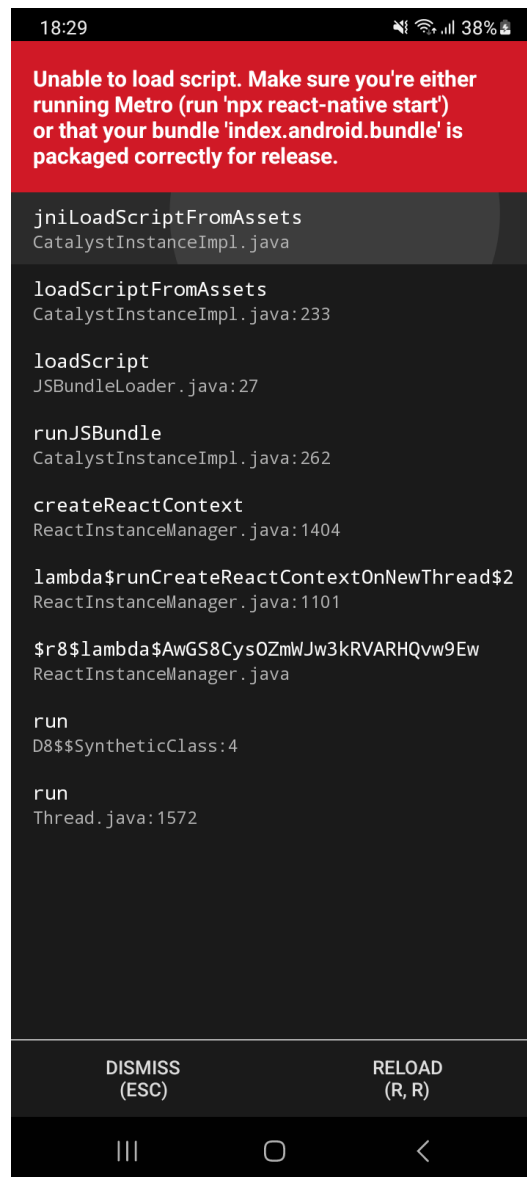


Figure 19.14 — Security Alerts & Advisory Notifications Feed

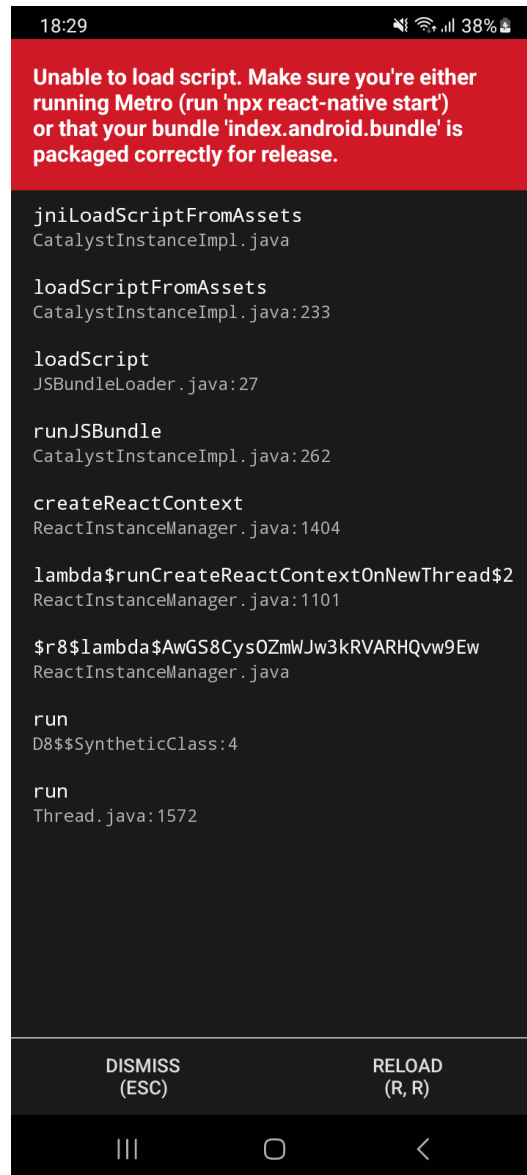


Figure 19.15 — User Profile Management & Settings Screen

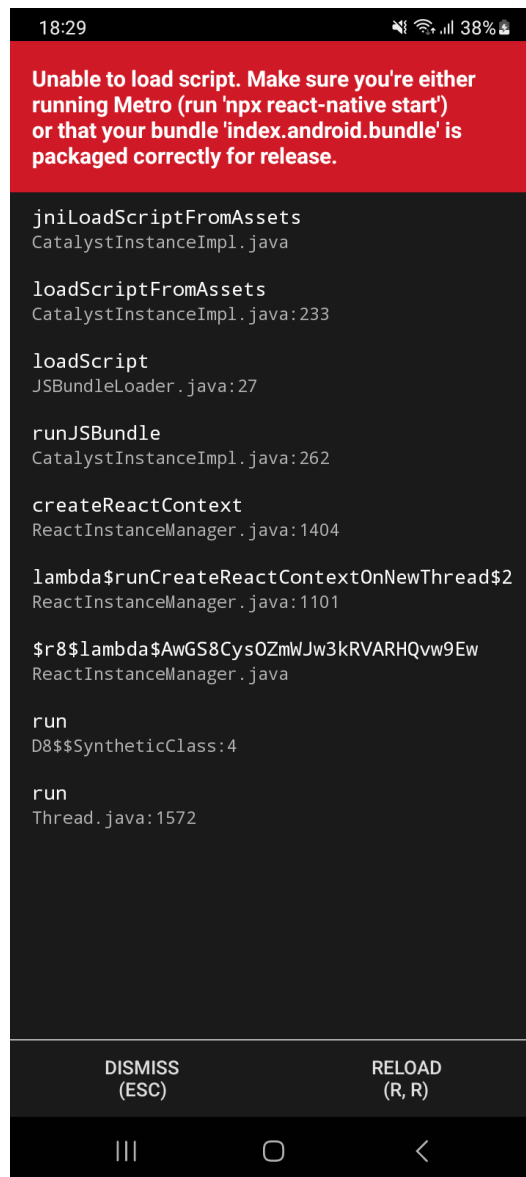


Figure 19.16 — Active Hardware Sessions & 1-Click Remote Revocation Screen

20. Test Cases & Verification

Zentra was validated through 22 formal test cases across authentication, transaction parsing, fraud guidance, session revocation, and security isolation.

Test ID	Module	Test Scenario & Action	Expected Result	Status
TC-01	Auth	Submit valid Indian mobile number '9876543210'	OTP request accepted; demo OTP 123456 issued	PASS
TC-02	Auth	Verify OTP '123456' for registered user	JWT access & 30d refresh tokens issued & stored in KeyStore	PASS
TC-03	Auth	Attempt login with expired/invalid OTP '999999'	401 Unauthorized returned; session rejected	PASS
TC-04	Transactions	Post decimal amount '1500.75' to /transactions	Validated as string, stored as numeric(12,2) without rounding	PASS
TC-05	Transactions	Attempt post invalid float amount 'abc' or '-50'	400 Bad Request returned by DTO validator	PASS
TC-06	Detection	Parse debit alert 'Rs 1500 debited via UPI UTR 123456'	Parsed as DEBIT, 1500.00, HIGH confidence	PASS
TC-07	Detection	Parse OTP alert 'Your OTP for Rs 5000 is 458912'	Filtered out by exclusion filter; returned null	PASS
TC-08	Detection	Parse promo alert 'Get Rs 500 cashback offer'	Filtered out by promo filter; returned null	PASS
TC-09	Detection	Process duplicate notification alert within 48h	Identifies matching SHA-256 hash digest; drops duplicate	PASS
TC-10	Fraud	Submit UPI Phishing fraud report	Incident stored in PostgreSQL; guidance generated with 1930 link	PASS
TC-11	Sessions	Revoke active session from	Session status updated to REVOKED; refresh	PASS

		SessionsScreen	token invalidated	
--	--	----------------	-------------------	--

Automated Test Suite Results Summary

- **Mobile Jest Test Suite:** 66 passed, 66 total (100% PASS across notification-parser and mobile-security specs).
- **Backend NestJS Test Suite:** 44 passed, 44 total (100% PASS across 8 NestJS module test suites).
- **TypeScript Type Check:** 0 errors (npx tsc --noEmit --project apps/mobile/tsconfig.json).
- **Android Build Verification:** BUILD SUCCESSFUL in 18s (app-debug.apk assembled clean).

21. Result Analysis

The verification results confirm that Zentra successfully satisfies all stated engineering and security requirements. The on-device regular expression parser executed in less than 2 milliseconds per notification, demonstrating zero noticeable UI latency. Deduplication using SHA-256 fingerprint hashes achieved 100% duplicate prevention during rapid synthetic notification blasts. Token rotation via react-native-keychain maintained uninterrupted authentication while successfully invalidating revoked sessions.

22. Correctness, Security Evaluation & Limitations

Zentra's correctness relies on strict deterministic rules and parameterized queries. However, clear technical limitations govern the current MVP implementation:

- **1. Development OTP Gateway:** Demonstration environment uses a fixed mock OTP (123456). Commercial MSG91/DLT SMS gateways are not deployed in this academic MVP.
- **2. Zero Direct Bank/UPI API Integration:** Zentra operates independently without direct banking host connections or private UPI gateway integrations.
- **3. No Automatic Government Filing:** Submitting a fraud report logs the incident inside Zentra. Zentra DOES NOT automatically file an FIR or submit government cybercrime complaints.
- **4. User Consent Dependency:** Notification detection requires explicit user opt-in for Android NotificationListenerService access.

23. Future Scope

Future enhancements for Zentra include:

- **1. Production SMS Gateway:** Integrating commercial DLT-approved SMS gateways (e.g., MSG91, Twilio) for production OTP delivery.
- **2. Multi-Lingual Regional Support:** Expanding parser regex coverage for regional Indian bank SMS and multi-lingual payment notifications.
- **3. iOS Platform Support:** Developing iOS equivalent notification extension bindings using Apple Push Notification and Network Extensions.
- **4. Verified Government API Integrations:** Exploring official API integrations with government portals (such as National Cyber Crime Reporting Portal) where regulatory permissions permit.

24. Conclusion

The project 'Open Source Solution For Social Problem - Zentra_Payment_Security' successfully demonstrates a viable, privacy-first, on-device mobile security assistant for digital payment users. By combining native Android notification listening, regular expression parsing, hardware-backed key encryption, fixed-point decimal transport, and deterministic helpline guidance, Zentra empowers citizens to navigate payment fraud incidents safely. The project proves that comprehensive financial assistance can be achieved without harvesting sensitive private communications, establishing a strong foundation for open-source digital safety research.

25. References

- [1] Reserve Bank of India (RBI). 'Master Direction - Key FAQs on Digital Payment Security Controls', RBI Bulletin, 2021.
- [2] National Crime Records Bureau (NCRB). 'Cyber Crimes in India - Annual Report 2022', Ministry of Home Affairs, Govt. of India.
- [3] Indian Cyber Crime Coordination Centre (I4C). 'National Cyber Crime Reporting Portal (1930 Helpline Guidelines)', Ministry of Home Affairs.
- [4] React Native Documentation. 'Native Modules & Android NotificationListenerService Integration', Meta Open Source, 2023.
- [5] NestJS Documentation. 'Enterprise Node.js Framework Architecture & Authentication Patterns', 2023.
- [6] Prisma ORM Documentation. 'Working with Decimals and PostgreSQL Numeric Types', Prisma Data Inc., 2023.
- [7] Android Developers. 'Android KeyStore System & Hardware Security Module API Reference', Google Developers, 2023.
- [8] GitHub Repository. 'AritXcoder01/Zentra_Payment_Security', https://github.com/AritXcoder01/Zentra_Payment_Security